Name : Harshad Ahire

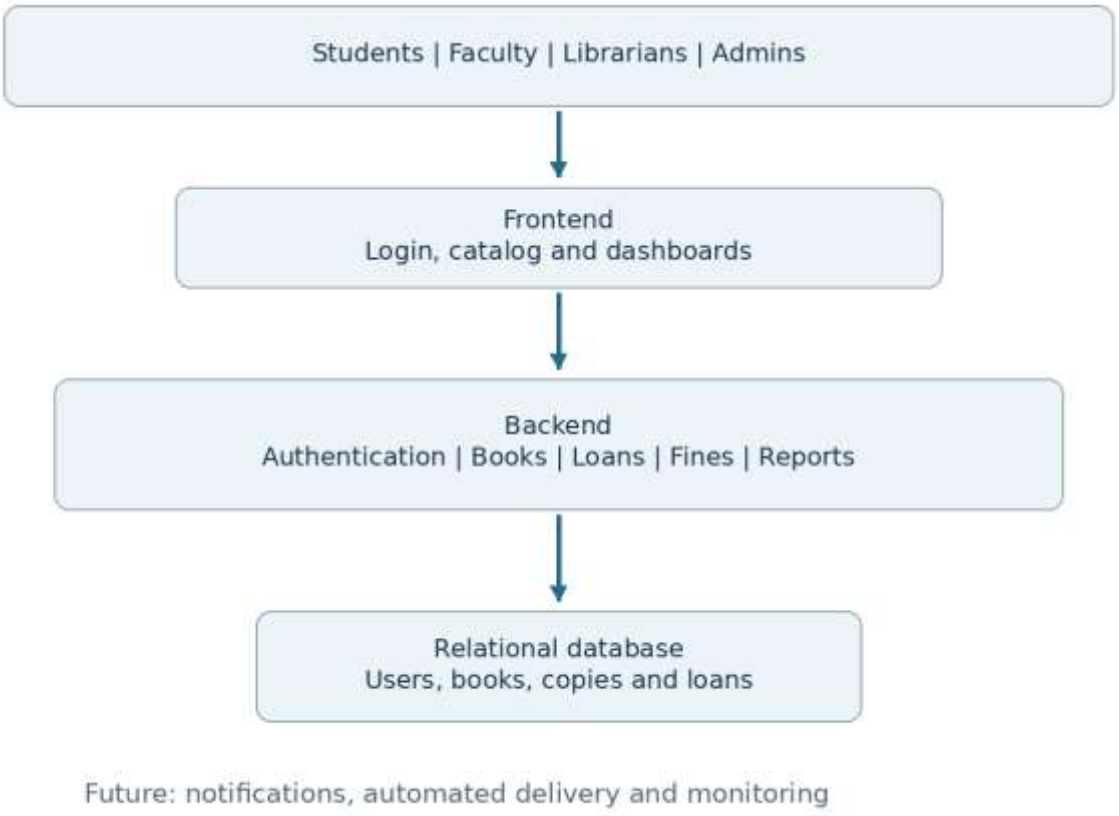
PRN : 2126UDSM2576 **Dept :** AIDS

SUBJECT : DEVOPS AND CLOUD FOUNDATION(IBM) MINI PROJECT

Library Management Platform

Unit 1

Product Engineering and Design Thinking



Project concept showing the proposed three layer system

Scope: Complete Unit 1 planning and design report. Application implementation and operational deployment remain future phases. Prototype images and worked examples use illustrative data.

Contents and project overview

1 Product engineering and SDLC

2 Library domain and business justification

3 Design Thinking and user research

4 Definition and solution selection

5 Prototype screens for members

6 Prototype screens for staff

7 Prototype testing and feedback

8 IBM Design Thinking and Loop

9 Personas and stakeholders

10 Functional requirements

11 Non functional requirements and rules

12 Product vision and MVP

13 Architecture and data design

14 Agile plan and engineering roadmap

15 Repository and execution guide

16 Verification and acceptance plan

17 Deliverables and presentation guide

Abstract

This project proposes a centralized Library Management Platform for an academic library. Members can search a catalog and see their loans. Librarians can maintain book records and process issues and returns. Administrators can manage access and review library activity. Unit 1 establishes the problem, users, requirements, prototype, architecture and delivery plan before coding begins.

Source basis: DevOps-Unit-1-Mini-Project-Guide-Doc.docx, supplied by the user and attributed within the guide to Careertiq Training and Engineering Division. Its text diagrams have been redrawn as readable figures. Additional screen designs, rules and test examples in this report are proposed project details.

1 Product engineering and SDLC

Programming means writing instructions that a computer can execute. Software product engineering covers the full process of understanding a problem, designing a useful product, developing it, testing it and maintaining it.

Aspect	Programming	Product engineering
Goal	Implement a function	Solve a user problem reliably
Work	Code and debugging	Research, design, code, testing and operations
Success	Correct execution	Usability, correctness and maintainability

Why software projects fail

Common causes are unclear requirements, selecting tools before understanding the problem, poor communication between teams and collecting feedback too late. This project reduces these risks with a clear MVP, documented rules and early prototype testing.



Fig 1 : Software development lifecycle redrawn from the guide

2 Library domain and business justification

An academic library acquires books, catalogs them, registers members, lends physical copies, records returns and monitors overdue items. A book title may have several physical copies, so the design tracks both titles and individual copies.

Existing process and proposed improvement

Operation	Possible manual difficulty	Proposed support
Cataloging	Inconsistent book information	Central catalog with title, author, ISBN and shelf
Membership	Membership details spread across registers	Member profile and borrowing rules
Search	Student must ask staff about availability	Search with copy availability
Issue and return	Entries may be missed or duplicated	Validated transactions and loan history
Overdue tracking	Manual date and fine calculation	Due date and fine calculation
Reporting	Repeated manual counting	Summary and overdue reports

Business justification

The proposed platform can reduce repetitive record keeping and make library information easier to access. A shared database helps staff maintain consistent records and gives members visibility into their own borrowing activity. These are expected benefits, not measured results.

How benefits will be evaluated

Measure the time needed to find an available book and complete an issue transaction before and after implementation. Check transaction accuracy by comparing a sample of physical copies with database records. Record overdue counts over comparable periods. The percentage improvements shown in the source guide are example targets and are not claimed as achieved here.

Project boundary

The MVP supports one academic library. Book purchasing, accounting, payment gateways, interlibrary lending, self checkout and AI recommendations are outside the first release.

3 Design Thinking and user research

Design Thinking is a way of solving problems by understanding users, defining their needs, exploring ideas, making prototypes and learning from feedback.

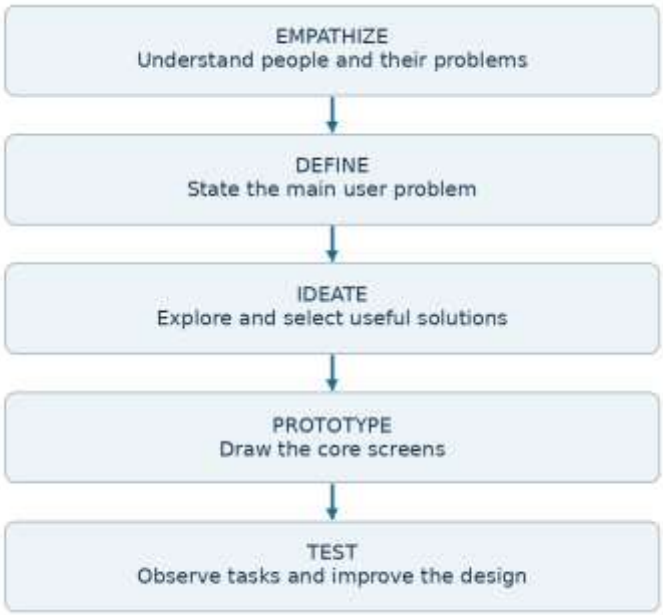


Fig 2 : The five stages of Design Thinking

Research plan

Interview one librarian, three students and one faculty member if available. Observe a book search and an issue or return with permission. Record the process without collecting passwords or unnecessary personal information.

Ask: How do you find a book? How do you know it is available? How are due dates communicated? How are fines calculated? Which task causes the most difficulty? What would make that task easier?

Evidence status: No real interviews or field observations are supplied with this report. The following personas, pain points and feedback examples are design assumptions to validate, not completed research.

4 Definition and solution selection

Problem statement

Library members may experience delays in locating available books and tracking due dates when information is managed manually. Library staff need a consistent way to maintain inventory, process borrowing and returns, and identify overdue loans. The proposed system brings these activities into one role based platform.

How might we question

How might we help members find an available book quickly while helping librarians maintain accurate borrowing records with fewer repetitive steps?

Proposed empathy map for a student

Area	Illustrative assumption
Says	I want to know if the reference book is available.
Thinks	Will I find the right book before my assignment deadline?
Does	Searches shelves or asks the librarian for help.
Feels	Frustrated when availability or due dates are unclear.

Idea selection

Idea	Priority	Reason
Search and availability	MVP	Addresses the main discovery problem
Issue and return	MVP	Core library operation
Automatic fine calculation	MVP	Reduces repeated date calculations
Role based access	MVP	Separates member and staff actions
Email reminders	Later release	Useful after circulation is stable
QR membership and barcode tools	Later release	Optional convenience integration
Mobile self checkout and AI suggestions	Future	Additional complexity outside Unit 1

Selected solution: A browser based system with separate member and staff views, backed by a shared database. Availability should be derived from copy records to avoid inconsistent manual totals.

5 Prototype screens for members

These are original static design prototypes. The buttons, search fields and sample records illustrate intended behavior; they are not screenshots of a running application.

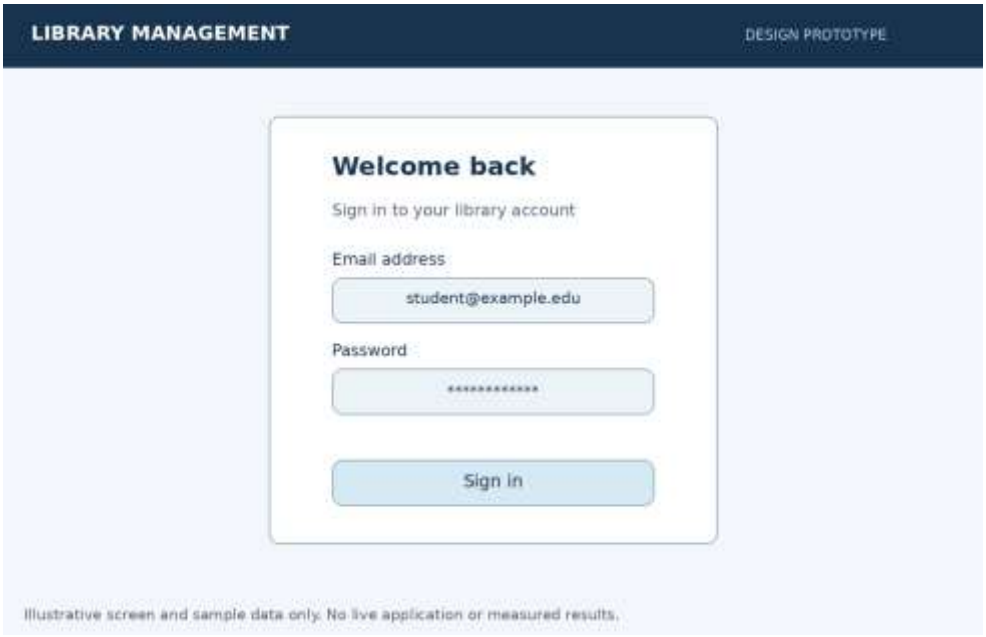


Fig 3 : Login screen with email and password

After successful login, the backend determines the account role. Users do not gain staff access by selecting a role on the screen. Invalid credentials produce a clear error message.

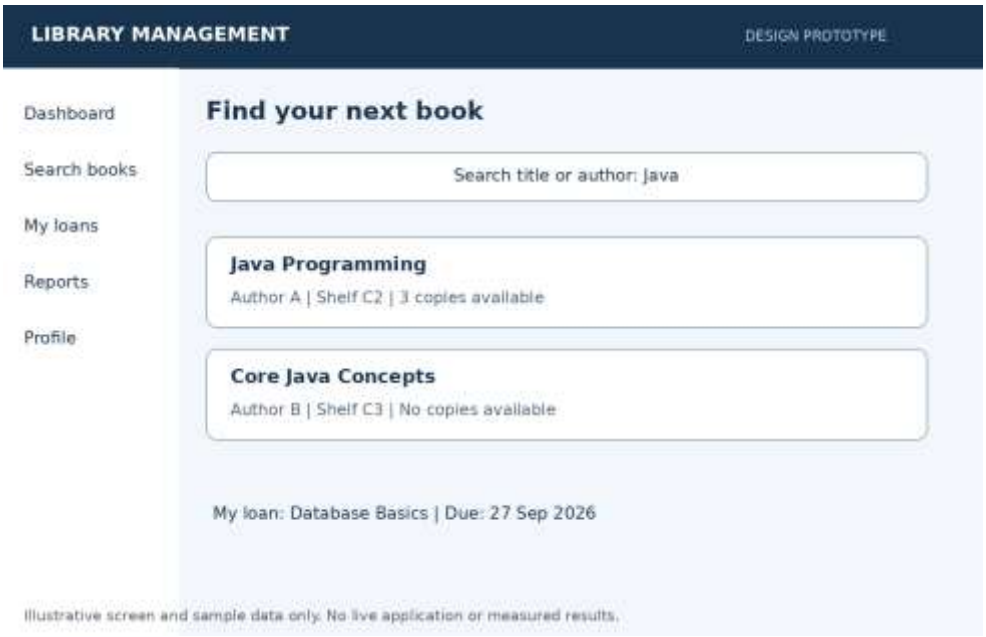


Fig 4 : Student search and borrowing overview

Members search by title or author, check available copies and view their own due dates. A zero result search should suggest another term. Faculty use the same core view with their assigned borrowing limits.

6 Prototype screens for staff

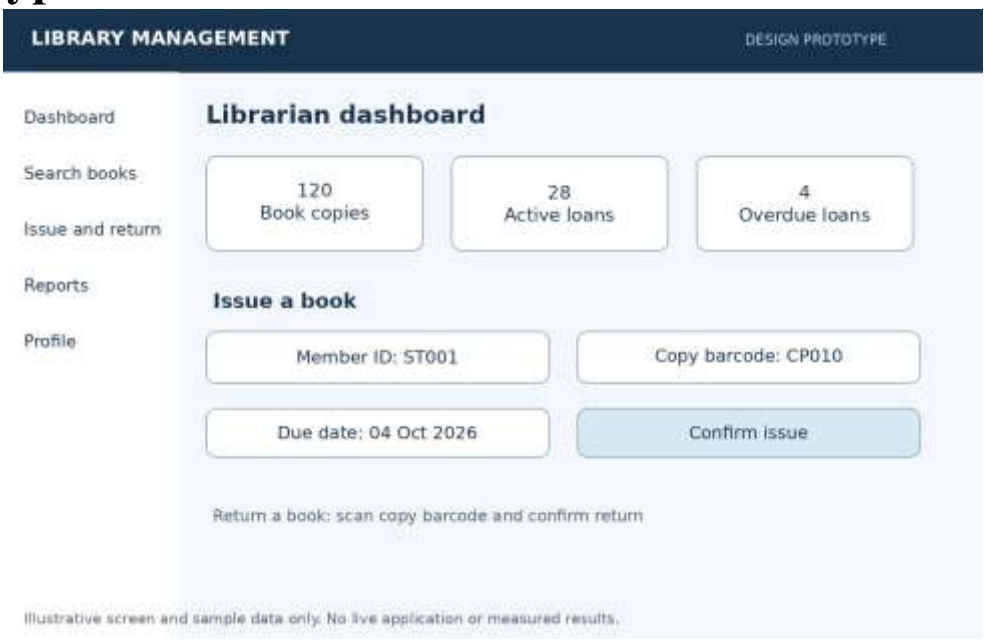


Fig 5 : Librarian issue and return dashboard

The librarian enters the member ID and physical copy barcode. The system checks membership, borrowing quota and copy status before issuing. A return closes the active loan and updates copy status in one transaction.

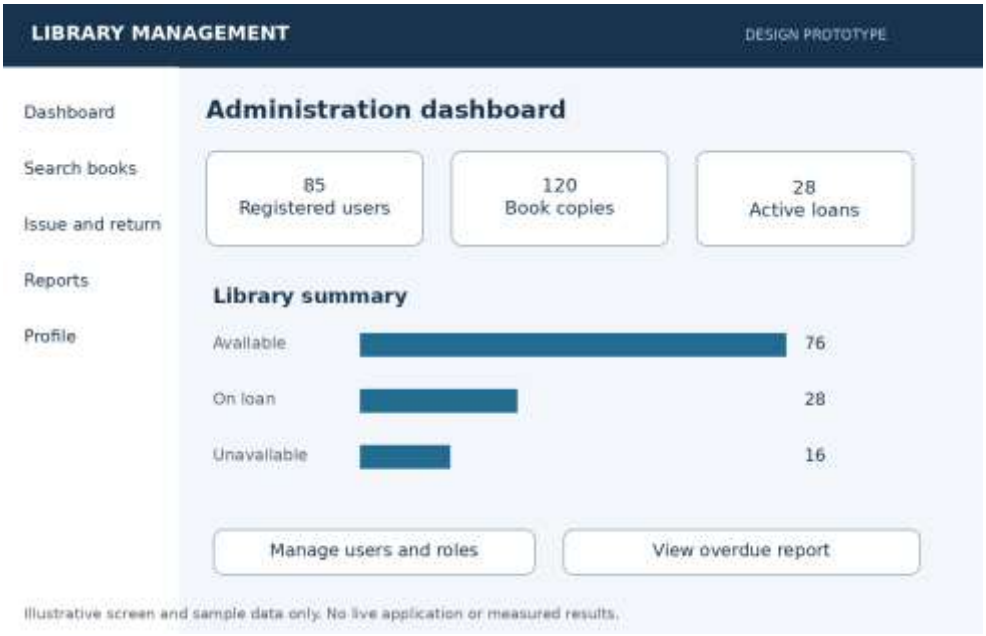


Fig 6 : Administrative summary and user management

The example totals are consistent: 76 available copies + 28 on loan + 16 unavailable copies = 120 copies. The 4 overdue loans in the librarian view are a subset of the 28 active loans. All numbers are illustrative.

7 Prototype testing and feedback

Purpose and method

Prototype testing checks whether users understand the screens before software is built. Show the screens to representative users, provide a task without explaining the answer, and observe the steps they choose. Record confusion, comments and completion time.

Task	Expected user action	Proposed success target
Find an available Java book	Use search and identify an available result	Complete within 60 seconds
Find a return deadline	Open own borrowing details	Identify the correct due date
Issue a copy	Enter member and copy IDs and confirm	Complete without missing a required field
Check overdue loans	Open the overdue report	Identify which loans are overdue

Illustrative feedback and response

Hypothetical feedback	Planned improvement
Availability is difficult to notice.	Show a clear text label next to each result.
I do not know which ID to enter.	Label member ID and copy barcode separately.
I cannot locate the due date.	Display it with every active loan.
I am unsure whether the return worked.	Show a confirmation after a successful return.

Actual evidence still required

Record the participant role, date, task, outcome, time, comment and resulting design change during real testing. Do not mark a task as passed until it has been observed. The examples above are a prepared test plan, not collected feedback.

Iteration

Revise the confusing screen, repeat the same task with users and compare the observations. Keep both versions of the screen so the report can explain what changed and why.

8 IBM Design Thinking and Loop

IBM Design Thinking emphasizes user outcomes, continuous improvement and collaboration across different roles. The project applies these ideas to library tasks rather than treating the number of screens or features as the main measure of success.

Principle	Application to this project
Focus on user outcomes	Help a student identify an available book and its shelf location with few steps.
Restless reinvention	Improve search and borrowing screens when observations reveal confusion.
Diverse empowered teams	Include members, librarians, developers, testers and operations responsibilities in decisions.

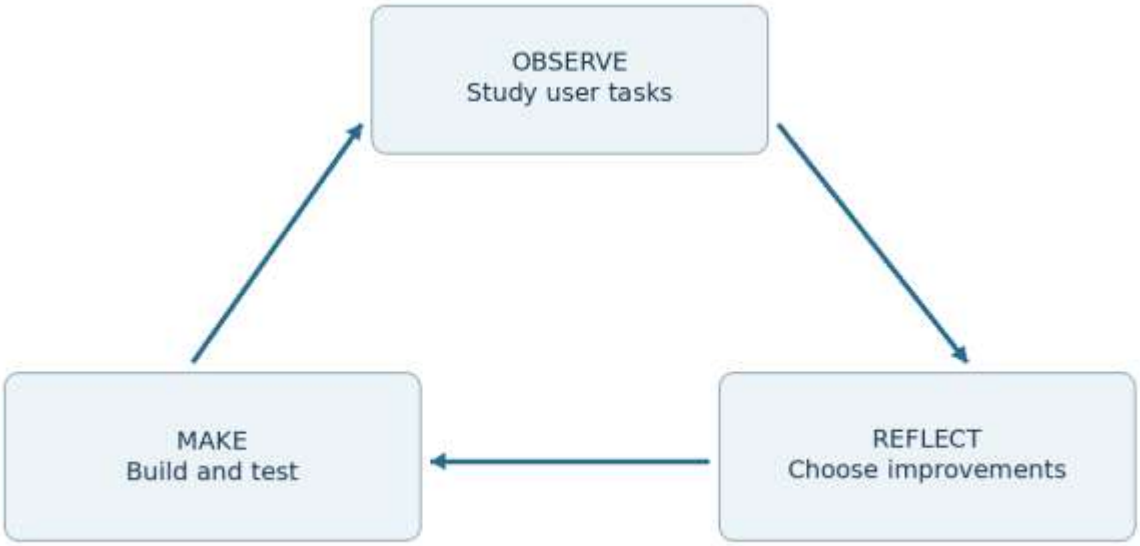


Fig 7 : IBM Observe Reflect Make loop redrawn from the guide

Worked example

Observe: In a future test, a member may struggle to notice the due date. Reflect: The team considers whether the date is too far from the book title. Make: Move the due date beside the title and test the revised screen. Repeat the cycle using actual observations.

Example outcome target: A member can locate an available book and its shelf position with no more than three main interactions after login. Validate the target during testing; it is not an achieved result.

9 Personas and stakeholders

The personas below are illustrative representatives based on the user categories in the source guide.

Personas	Goal	Difficulty	Design response
Aarav Student	Find academic references	Unclear availability and due dates	Search, availability and own loan view
Sunita Librarian	Maintain accurate lending records	Repeated entries and copy discrepancies	Validated issue and return workflow
Dr Verma Administrator	Understand library usage	Slow manual reporting	Summary and overdue reports
Faculty member	Borrow teaching references	Unclear borrowing allowance	Role specific policy and loan history

Stakeholder responsibility matrix

Group	Responsibility	Expected contribution
Students and faculty	Validate member needs	Review search and loan screens
Librarian	Define circulation rules	Confirm fields, quotas and workflows
College management or library admin	Approve scope and access policy	Review MVP and reports
Developer and UI designer	Design and implement the product	Maintain code and usable interfaces
QA responsibility	Verify expected behavior	Run functional and access tests
DevOps and database responsibility	Plan delivery and data reliability	Automate checks and validate backup recovery

In a student mini project, one person may handle several engineering responsibilities. Stakeholder roles describe accountability; they do not claim that a large project team has been assigned.

10 Functional requirements

A functional requirement states a behavior that the system must support. Each requirement below includes a verification condition so future implementation can be checked objectively.

ID and requirement	Expected behavior and acceptance condition
FR01 Authentication and access	Registered users can log in. The server enforces Admin, Librarian, Student and Faculty permissions. A member cannot issue books or access another member's loan history.
FR02 Catalog management	Staff can create, view, edit and retire book records, authors, categories and shelf details. Required fields must be validated. Records linked to loan history must not be destructively deleted.
FR03 Search and availability	Members can search by title or author and view availability and shelf information. Unknown search terms return a clear no results message.
FR04 Circulation	Staff can issue an available copy to an eligible member and return an active loan. Copy state and loan changes are atomic. A copy cannot have two active loans.
FR05 Fine calculation	The system calculates overdue charges using the applicable daily rate. Fine is zero on or before the due date and stops increasing after return.
FR06 Reports	Admin and authorized staff can view total copies, active loans and overdue loans. Available counts agree with the copy records.
FR07 Notifications	A later release sends due date and overdue email reminders. Delivery failures are logged and retried without uncontrolled duplicate messages.

Access matrix

Action	Student or Faculty	Librarian	Admin
Search catalog	Yes	Yes	Yes
View own loans	Yes	Yes	Yes
Manage books and circulation	No	Yes	Yes
View library reports	No	Yes	Yes
Manage users and roles	No	No	Yes

11 Non functional requirements and rules

Non functional requirements describe qualities such as security, speed and reliability. These are design targets to verify during later development.

Quality	Proposed requirement	Verification approach
Security	Hash passwords; enforce server side authorization; protect deployed traffic with HTTPS. JWT sessions are proposed in the guide.	Inspect credential storage and run role access tests.
Performance	Guide target: search API response below 500 ms. Proposed measurement: 95th percentile with 20 concurrent users and 1000 titles.	Run a repeatable load test and record environment and results.
Availability	Guide target: 99.9% production uptime; not a local prototype guarantee.	Measure successful service availability over an agreed window.
Maintainability	Separate modules, documented configuration and version controlled infrastructure.	Review structure and setup instructions.
Data reliability	Atomic circulation updates and recoverable backups.	Test duplicate issue prevention and restore a backup.
Usability	Readable screens, clear labels and useful errors.	Observe member and staff tasks.

Proposed borrowing policy requiring library approval

Student: up to 3 active loans for 14 days. Faculty: up to 5 active loans for 30 days. Example daily rates: student ₹2 and faculty ₹1 per overdue day. These values are project assumptions, not college rules. Use the library's local calendar date and no grace period for this example.

Overdue days = max(0, effective date − due date). Effective date is the return date for a closed loan and today for an open loan. Fine = overdue days × daily rate captured when issuing the copy.

Example: A student copy due on 10 September and returned on 13 September is 3 days overdue. At ₹2 per day, the fine is ₹6. Returning it on 10 September gives a fine of ₹0.

12 Product vision and MVP

Product vision

To provide an easy to use academic Library Management Platform that gives members clear book information and helps staff maintain accurate circulation records with less repetitive work.

MVP feature matrix :

Capability	First release	Later expansion
Login and profiles	Role based access and basic profile	Advanced account recovery
Catalog	Title and author search, availability and shelf	Advanced filters and recommendations
Circulation	Issue, return and borrowing history	Reservations, renewals and self checkout
Fines	Automatic calculation	Payment collection integration
Reports	Basic counts and overdue report	Detailed trends and PDF export
Notifications	Outside MVP	Due date and overdue email alerts
Operations	Local setup and planned automated tests	Cloud delivery and operational monitoring

Definition of done for the future MVP

The core workflows operate correctly, required access checks pass, automated checks complete successfully, setup documentation is usable, and a reviewer can search, issue and return a sample copy. Known limitations are documented. This report alone does not establish that the software meets these conditions.

Risks and mitigations :

Risk	Mitigation
Scope grows beyond available time	Keep the agreed MVP and defer optional features.
Wrong fine or borrowing policy	Confirm rules with the librarian before coding.
Duplicate issue or mismatched inventory	Use transactions and enforce one active loan per copy.
Loss of borrowing records	Back up data and verify a restore.
Confusing screens	Test prototypes and iterate before implementation.

13 Architecture and data design

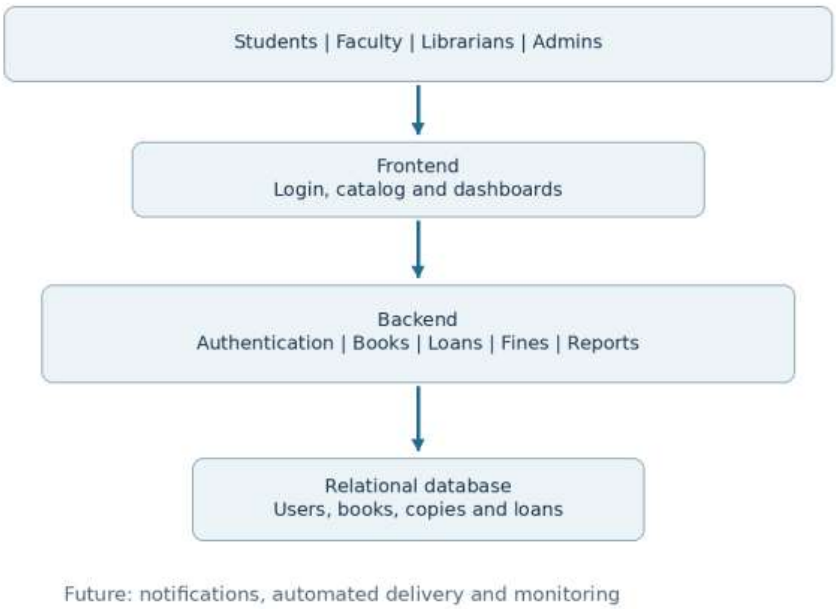


Fig 8 : Proposed logical system architecture

The frontend sends requests to the backend. The backend validates identity, permissions and business rules before reading or updating the relational database. The database is not exposed directly to the browser. Notifications are a later extension.

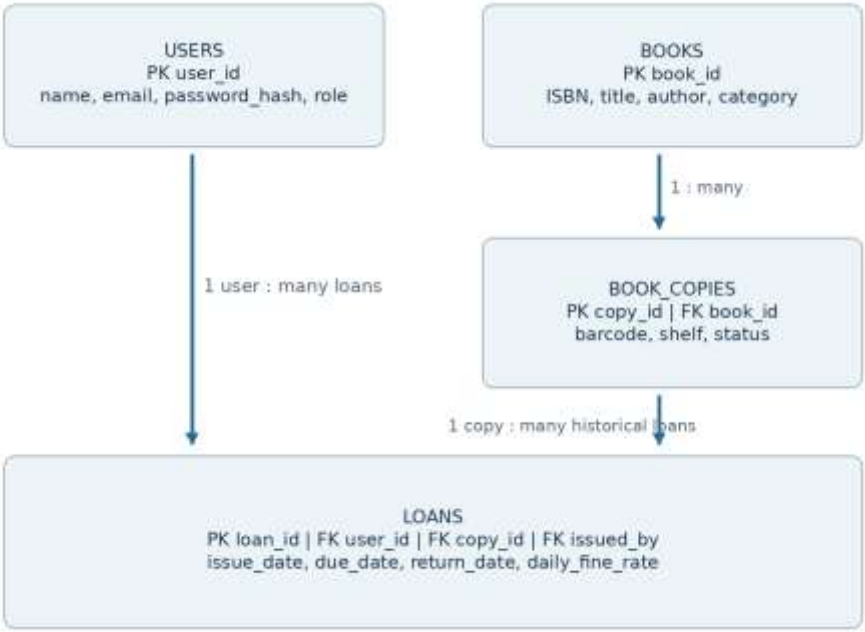


Fig 9 : Core entity relationship design

PK means primary key, a unique row identifier. FK means foreign key, a link to another table. A book has many physical copies; each copy can have many historical loans but at most one active loan. issued_by identifies the staff user who processed the issue.

14 Agile plan and engineering roadmap

Agile delivery breaks development into small increments that can be reviewed. The following is a proposed four sprint MVP plan; actual dates depend on the academic schedule.

Sprint	Main work	Review output
1	Validate requirements, design database, implement login and roles	Role controlled login and approved schema
2	Implement catalog and search	Searchable sample catalog and copy availability
3	Implement issue, return and fine rules	Validated circulation demonstration
4	Add basic reports, verify workflows and document setup	Reviewed local MVP and automated checks

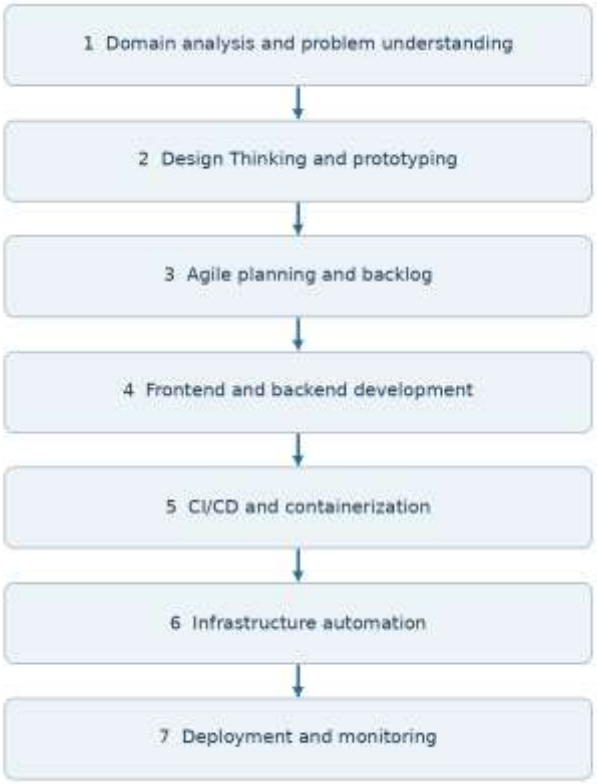


Fig 10 : Multi phase roadmap redrawn from the guide

Unit 1 produces the domain and design deliverables and the initial repository structure. Later phases add implementation, CI/CD, Docker, Terraform, Ansible, Kubernetes, Prometheus and Grafana as required by the course. These tools are roadmap items, not completed deployment evidence.

15 Repository and execution guide

Use the source guide's repository layout. Each directory should contain the stated material when its phase begins. Empty future folders may contain a short README describing their planned purpose.

Path	Purpose
app/modules/	Books, users, loans and fine logic
app/database/	Schema, migrations and database models
app/tests/	Automated unit and integration tests
docs/	Report, requirements, diagrams and prototype images
.github/workflows/	Future GitHub Actions workflows
docker/	Future container definitions and Compose setup
kubernetes/	Future deployment and service manifests
terraform/	Future cloud infrastructure definitions
ansible/	Future configuration automation
monitoring/	Future metrics and dashboard configuration
README.md	Project objective, scope and setup instructions

PowerShell starter commands :

Run in a location where you want a new project folder. Git must be installed. These commands are instructions for the student and are not evidence of execution on the student's computer.

```
mkdir library-management-platform
cd library-management-platform
git init
$projectDirs = @("app/modules", "app/database", "app/tests",
  "docs", ".github/workflows", "docker", "kubernetes",
  "terraform", "ansible", "monitoring")
$projectDirs | ForEach-Object {
  New-Item -ItemType Directory -Force -Path $_
  Set-Content -Path "$_/README.md" -Value "Planned project area."
}
Set-Content README.md "# Library Management Platform"
git add .
git commit -m "Initialize Unit 1 project structure"
git status
```

If Git asks for an identity, configure your own name and email, then repeat the commit. Save the report and diagrams under docs and commit them. Capture your real folder view, commit output and git status for submission evidence.

16 Verification and acceptance plan

The following tests are prepared for implementation. All are currently Not executed because this deliverable is a planning and design report.

Test	Scenario	Expected result
T01	Correct and incorrect login credentials	Correct credentials succeed; incorrect ones are rejected.
T02	Student requests another member’s loans	Access denied by the server.
T03	Search for known and unknown titles	Matching results or a clear no results message.
T04	Issue an available copy within quota	One active loan is created; copy becomes on loan.
T05	Two staff issue the same copy concurrently	Only one request succeeds.
T06	Member already at borrowing limit	Issue is rejected with a clear reason.
T07	Return an active loan	Return date is saved; the copy becomes available.
T08	Repeat the same return action	No duplicate stock increase or second return record.
T09	Return on due date and three days late	Fine is zero and three times the captured daily rate.
T10	Compare reports with fixture records	Counts match the known sample dataset.
T11	Restore a database backup	Users, copies and loan history are recoverable.
T12	Run search performance test	Record measured response time against the stated target.

Suggested demonstration sequence

Log in as a student, search for a book and check availability. Log in as a librarian, issue one copy, then show its updated status. Return the copy and show the resulting fine using controlled dates. Finally, open the admin summary and explain how counts are derived.

The four images in this report can be used for a Unit 1 walkthrough. A working software demonstration should only be claimed after the application has actually been implemented and tested.

17 Deliverables and presentation guide

Required deliverable	Location or status
Business problem and justification	Sections 2 and 4
Vision and MVP feature matrix	Section 12
Personas and stakeholder matrix	Section 9
Functional and non functional requirements	Sections 10 and 11
Design Thinking report and prototype images	Sections 3 to 7
IBM principles and Loop	Section 8
Engineering roadmap	Section 14
Architecture and database concept	Section 13
Repository structure and initialization	Section 15 provides commands; student execution evidence remains pending
Actual interview and prototype observations	Pending field validation; examples are explicitly labeled